

Sustainable Software Engineering - Accountability, Transparency & Governance



Sustainable Software Engineering: Who Answers for Biased Software?

This question introduces the need for **accountability** — the idea that someone must be responsible for the ethical consequences of software decisions. Fairness is important, but **fairness alone is not sustainable** unless there are structures ensuring it is maintained and enforced.

This question highlights that **ethical responsibility** is central to **social sustainability**. A society cannot sustain trust in digital technologies if accountability is absent.

Real-world cases illustrate this:

- **COMPAS (2016):** A U.S. judicial algorithm falsely flagged Black defendants as higher risk, with no mechanism for oversight.
- **Amazon Hiring Algorithm (2018):** Penalized resume containing “women’s.”
- **Facial Recognition (2018–2020):** Higher misidentification rates for women and darker-skinned people.

Each example shows how *ethical failure undermines social sustainability* by eroding public trust, fairness, and inclusiveness — key pillars of sustainable societies.

Sustainable Software Engineering: Accountability

Accountability means being answerable for decisions, outcomes, and their societal impact. In software systems, it bridges **ethical responsibility** and **practical governance**.

Floridi et al. identify accountability as one of the core pillars of trustworthy AI — the mechanism that ensures moral values are preserved through real-world structures.

For a software system to be sustainable, it must sustain trust, fairness, and inclusivity over time.

Accountability transforms these ethical principles into **maintainable processes**, enabling both long-term compliance and moral integrity.

Sustainable Software Engineering: Accountability

Accountability means that individuals, teams, and organizations are *answerable for the consequences* of the software systems they build or deploy.

In practice, accountability involves:

1. **Attribution** — identifying who made or approved a decision.
2. **Justification** — explaining why that decision was made.
3. **Evaluation** — assessing whether the decision aligns with ethical and legal norms.

In software systems — especially those using machine learning — accountability ensures that when harm occurs, there is a **traceable line** connecting the outcome to specific design, data, or deployment choices.

Sustainable Software Engineering: Example - Credit Scoring Algorithm

Imagine a company deploying a **machine learning credit scoring model** that predicts whether users are likely to repay loans.

- **Scenario:** After launch, journalists reveal that the model gives *lower scores to applicants from certain postal codes* — areas with higher proportions of ethnic minorities.
- **Investigation:** Developers claim the algorithm uses “neutral” data, but further analysis shows that postal code correlates with ethnicity and income due to historical segregation.
- **Accountability Breakdown:**
 - There was **no audit trail** explaining who approved the use of postal code as a feature.
 - There was **no fairness assessment** before deployment.
 - The organization lacks a **responsible AI officer** or internal ethics process.

As a result, when bias is detected, *no one can clearly answer who is responsible or how the decision was made.*

This represents a failure of **accountability**, undermining **social sustainability** by eroding public trust and **organizational sustainability** by exposing the company to legal and reputational risks.

Sustainable Software Engineering: Relantio with Sustainability

Accountability is not just a moral expectation — it's a *sustainability mechanism*. It enables systems to remain **ethical, trustworthy, and resilient** over time by creating feedback loops for continuous reflection and improvement.

Sustainability Aspects	How Accountability Supports It
Ethical sustainability	Ensures that design and development choices uphold values like fairness, inclusivity, and respect for human rights.
Social sustainability	Builds long-term public trust by making responsibility transparent and enforceable.
Technical sustainability	Encourages better documentation, testing, and monitoring — improving maintainability and debuggability.
Organizational sustainability	Reduces ethical risk and reputational damage, ensuring that processes outlive individual employees.

To make accountability operational in software projects, teams can:

- Keep a **decision log** documenting key design choices and ethical trade-offs.
- Assign **responsibility roles** for model validation and impact assessments.
- Use **audit trails** for data preprocessing and model updates.
- Conduct **post-deployment reviews** assessing social impact.

Sustainable Software Engineering: Dimensions of Accountability

Accountability operates across three interconnected domains:

1. **Moral Accountability:** The ethical duty of individuals and teams to prevent harm. Developers must critically assess how design decisions affect users.
 - Example: Refusing to use data that reinforces racial or gender stereotypes.
 - Sustainability link: Promotes social justice and long-term societal well-being.
2. **Legal Accountability:** The responsibility enforced by laws, regulations, and professional standards.
 - Example: The EU AI Act (2024) requires high-risk AI systems to maintain documentation and allow human oversight.
 - Sustainability link: Provides institutional continuity and protects public interests.
3. **Organizational Accountability:** Processes that embed ethical reflection into workflows — audits, documentation, review boards, and feedback mechanisms.
 - Example: Microsoft's "Responsible AI Standard" incorporates fairness checks in product pipelines.
 - Sustainability link: Prevents ethical decay as organizations grow and evolve.

Sustainable Software Engineering: Accountability and SSE

Accountability transforms fairness from a static property into a **continuous process**. It aligns with **sustainable software engineering**, where systems are designed to remain ethically and socially stable as data, teams, and contexts change.

Sustainable accountability requires:

- Mechanisms for feedback and correction.
- Transparent communication channels.
- Periodic audits and ethical reviews.

Like environmental sustainability, **social sustainability** involves monitoring and adaptation. Systems that can self-assess and evolve ethically over time contribute to a socially resilient technological ecosystem.

Sustainable Software Engineering: Transparency as a Foundation for Trust

In software and AI systems, **transparency** refers to the *degree to which the inner workings of a system are visible, understandable, and open to scrutiny*.

A transparent system allows stakeholders — developers, users, regulators, or the public — to see **how** and **why** the system makes decisions.

Transparency is both **an ethical principle** and a **technical property**:

- **Ethical**, because it empowers people to challenge, audit, and understand the systems that affect their lives.
- **Technical**, because it requires clear documentation, interpretable code, traceable data, and reproducible results.

In sustainable software engineering, transparency is essential because **systems cannot be trusted, improved, or maintained without visibility**.

Transparency enables **continuous ethical verification**, just as logging or version control enables continuous technical reliability.

Sustainable Software Engineering: Transparency as a Foundation for Trust

Users are more likely to accept and rely on technology when they can understand its behavior, limitations, and rationale.

When systems are opaque — or "black boxes" — users and regulators lose confidence, even if the system performs well.

EXAMPLE

In 2020, the UK government used an algorithm to assign A-level grades after exams were canceled due to COVID-19.

- The algorithm reduced grades for students from lower-income schools.
- The decision-making process was opaque — no one outside the development team could see how the model worked.
- Public outrage led to the model's withdrawal and a loss of trust in both government and data-driven systems.

This case demonstrates that **a lack of transparency can directly harm social trust and sustainability.**

Sustainable Software Engineering: Transparency as a Sustainable Property

Transparency contributes to multiple **dimensions of sustainability**:

Sustainability Aspects	Role of Transparency	Outcome
Ethical sustainability	Makes biases and ethical trade-offs visible.	Ensures moral values can be monitored and upheld over time.
Social sustainability	Builds trust and legitimacy among users and communities.	Encourages long-term acceptance of technology.
Technical sustainability	Supports debugging, maintenance, and model reproducibility.	Extends system lifespan and reliability.
Organizational sustainability	Allows effective audits and compliance reviews.	Reduces risk and supports continuous accountability.

Transparency is thus the *precondition* for a sustainable relationship between technology and society — it keeps systems open to inspection and correction.

Sustainable Software Engineering: Example from Software Engineering Practice

GitHub Copilot, an AI-powered code suggestion tool, faced criticism for generating code snippets taken from open-source projects without clear attribution.

Transparency Issues:

- Users were not initially informed about which data Copilot's models were trained on.
- The system provided no visibility into whether generated code contained license-protected material.

Sustainability Consequences:

- **Ethical sustainability:** Threatened, as developers couldn't verify compliance with licensing ethics.
- **Social sustainability:** Distrust grew in the open-source community.
- **Technical sustainability:** Lack of traceability made it harder to fix biased or non-compliant outputs.
- **Organizational sustainability:** GitHub faced reputational and legal challenges.

Outcome:

GitHub improved transparency by releasing documentation on training data sources and guidance for safe code usage — restoring partial trust and enabling a more **sustainable collaboration** between AI systems and human developers.

Sustainable Software Engineering: Transparency in SSE

Transparency is not only an ethical concern but a **core engineering principle** for sustainability.

It aligns directly with several goals of **Sustainable Software Engineering (SSE)**:

- 1. Traceability:** Transparent systems leave logs and documentation that record data provenance, model updates, and design decisions — enabling accountability and reproducibility.
- 2. Maintainability:** Clear documentation and open interfaces reduce “knowledge loss” over time, making systems easier to maintain sustainably.
- 3. Auditability:** Transparency allows both internal and external actors to evaluate system performance and compliance.
- 4. Continuous Improvement:** Transparency supports ethical monitoring — detecting bias, drift, or misuse across a system’s lifecycle.
- 5. Stakeholder Inclusion:** Transparency enables *participatory design* — involving users, domain experts, and regulators in understanding and improving systems.

In short, **transparency is the sustainability mechanism that connects technical robustness to ethical responsibility.**

Sustainable Software Engineering: The Layers of Transparency

Transparency exists at multiple interdependent layers:

- **Data Transparency:** Documentation of dataset sources, collection conditions, and potential biases.
- **Model Transparency:** Clarity on model architecture, training logic, and decision rules.
- **Decision Transparency:** Explanations for individual outputs (e.g., why a loan was denied).
- **Impact Transparency:** Awareness of how systems affect individuals and communities.

Each level reinforces **social and technical sustainability** by making bias detection and correction possible across a system's lifecycle.

Sustainable Software Engineering: Explainability - Making AI Understandable

Transparency reveals; explainability interprets. **Explainability** refers to the ability to present model behavior in human-understandable terms.

It allows users and stakeholders to comprehend, evaluate, and challenge system decisions — essential for maintaining fairness over time.

Techniques and examples:

- **LIME / SHAP:** Identify which features influenced an AI decision.
- **Counterfactuals:** Show how small changes (e.g., higher income) would alter an outcome.
- **Example:** In credit scoring, explainability clarifies that a loan rejection is due to insufficient repayment history, not demographic factors.

Explainability ensures **social sustainability** by empowering users and regulators to hold systems accountable

Sustainable Software Engineering: The EU AI Act (2024) and Sustainable Regulation

The **EU Artificial Intelligence Act (AI Act)**, adopted in 2024, is the world's **first comprehensive legal framework for AI and automated decision-making systems**.

It seeks to ensure that AI technologies used in the European Union are **safe, transparent, accountable, and aligned with human rights** — while supporting innovation.

Its overarching principle is “**trustworthy and human-centered AI**.”

Unlike voluntary ethics guidelines, the AI Act is *legally binding*, establishing enforceable standards that developers, organizations, and importers of AI systems must meet.

This legislation turns abstract ethical principles — *fairness, transparency, accountability* — into concrete obligations. It therefore represents a crucial step toward **institutional and social sustainability** in software and AI governance.

Sustainable Software Engineering: The AI Act and SSE

The EU AI Act directly reinforces **Sustainable Software Engineering principles** by embedding ethical, social, and technical sustainability into the development lifecycle:

SSE Principle	AI Act Mechanism	Sustainability Benefit
Maintainability	Continuous documentation, audit trails, and post-market monitoring	Ensures system integrity and long-term adaptability
Transparency	Required disclosures and user information	Builds durable user trust and societal legitimacy
Fairness and Inclusivity	Data quality and bias control requirements	Reduces discrimination, supports equitable access
Human-Centric Design	Mandatory human oversight	Maintains human agency and ethical resilience
Accountability and Governance	Legal responsibility for system outcomes	Encourages organizational learning and institutional continuity

Thus, the Act provides a *legal infrastructure for sustainability* – ensuring that AI systems evolve responsibly and ethically across their entire lifecycle.

Sustainable Software Engineering: Automated Hiring System under the EU AI Act

Scenario:

A company deploys an AI-driven hiring tool that screens job applications using resume data and past hiring patterns.

Under the EU AI Act, this system is “high-risk” because it affects access to employment — a fundamental right.

Regulatory implications:

- The company must provide **dataset documentation** showing that training data are balanced across gender, ethnicity, and age.
- The model must be **explainable** — e.g., it should show that a candidate was ranked lower because of missing skills, not gendered terms.
- **Human reviewers** must remain in the decision loop.
- The company must maintain **post-deployment logs** to detect bias drift.

Sustainability Connection:

By institutionalizing these processes, the Act ensures that fairness and transparency are *continuously sustained* through structured review and feedback.

This turns accountability into a **living process** rather than a one-time compliance exercise — the essence of **sustainable software governance**.

Sustainable Software Engineering: Governance Frameworks and Responsible AI

Governance in software and AI refers to the **formal structures, rules, and procedures** that ensure ethical principles — such as fairness, accountability, transparency, and human oversight — are systematically embedded in the lifecycle of technology.

If *accountability* is about “who is responsible,” then *governance* is about “**how responsibility is organized and maintained.**”

Governance translates moral intentions into **institutionalized practices**:

- Defining **roles** (e.g., data steward, ethics officer, responsible AI lead)
- Establishing **policies** (e.g., risk assessment protocols, bias audits)
- Enforcing **processes** (e.g., documentation, third-party audits, compliance reviews)

This transformation is crucial for **sustainability** — because sustainable ethics cannot depend on individual goodwill; they require **durable systems** that persist over time.

Sustainable Software Engineering: Major Governance Frameworks

1. **AI4People (Floridi et al., 2018):**

- Principles: *Beneficence, Non-maleficence, Autonomy, Justice, Explicability.*
- Focus: Ethical sustainability and societal benefit.

2. **OECD Principles on AI (2019):**

- Emphasize human-centered, fair, accountable AI.
- Promote **global social sustainability** through shared ethical norms.

3. **EU AI Act (2024):**

- Legally enforces transparency, human oversight, and documentation.
- Ensures **institutional and organizational sustainability** by embedding ethics in law.

Together, these frameworks form a layered structure where ethics evolve from voluntary ideals to enforceable standards.

Example:

Google's internal *AI Principles* guide ethical reviews across products.

Microsoft's "Responsible AI Office" institutionalizes audits for fairness and bias before deployment.

These systems promote **organizational sustainability**, ensuring ethical consistency even as teams or technologies change.

Sustainable Software Engineering: Documentation as Ethical Infrastructure

Documentation in software and machine learning is more than technical writing — it's an **ethical instrument**.

It records *how* a system was built, *why* decisions were made, *what* data were used, and *who* is responsible.

Good documentation transforms invisible ethical assumptions into **visible, verifiable artifacts**.

It allows auditors, regulators, and future engineers to understand not only *what works*, but *why it works that way*.

In this sense, documentation is the **infrastructure of accountability** — the connective tissue linking people, data, and systems across time.

Sustainable Software Engineering: Why Documentation Matters for Sustainability

Without documentation:

- Fairness analyses cannot be reproduced.
- Biases in datasets or models are forgotten with team turnover.
- Responsibility is diffused and unsustainable.

With documentation:

- Ethical intent becomes durable.
- Systems can be monitored and improved responsibly.
- Social and technical trust are preserved.

Sustainability Aspect	Documentation Contribution
Ethical	Makes intentions and limitations explicit, enabling value alignment.
Social	Builds transparency and public confidence.
Technical	Ensures reproducibility, maintainability, and debuggability.
Organizational	Transfers knowledge across teams and generations.

Sustainable Software Engineering: Model Cards: Sustainable Model Transparency

Model Cards (Mitchell et al., 2019) are standardized documents describing the behavior, intended use, limitations, and ethical considerations of machine-learning models. They turn abstract fairness principles into **structured, inspectable information**.

A typical Model Card includes:

- Model details (name, version, training algorithm)
- Intended use and users
- Evaluation metrics (performance & fairness)
- Ethical considerations (risks & limitations)
- Caveats and recommendations

This mirrors *nutritional labeling* — giving consumers knowledge to use a product responsibly.

Model Cards embed **continuous reflection** into engineering: every new version must justify changes.

Over time, this creates an **evolving ethical record**, essential for sustainable software development and regulation (e.g., compliance with the EU AI Act's documentation duties).

Sustainable Software Engineering: Datasheets for Datasets: Sustainable Data

Datasheets (Gebru et al., 2018) apply the idea of **documentation accountability** to datasets.

Just as electronic components include datasheets describing voltage limits, AI datasets should describe *composition*, *collection context*, and *bias risks*.

A Datasheet typically covers:

1. Motivation and intended use
2. Composition and size
3. Collection process and ethics
4. Preprocessing steps
5. Recommended and inappropriate uses
6. Distribution and maintenance

By documenting origins and context, Datasheets make data pipelines **transparent, reusable, and ethically traceable**.

Datasheets embed sustainability not just in code but in **data provenance** — the foundation of all ML systems.

Sustainable Software Engineering: Integration into SSE Practical strategies:

Practical strategies:

1. Include Model Cards and Datasheets as CI/CD artifacts — generated or updated with each model build.
 2. Store ethical metadata in version control (e.g., Git repositories with ethical CHANGELOGs).
 3. Automate checks for missing documentation before deployment.
 4. Link documentation to post-market monitoring dashboards for continuous feedback.
- Those embed **accountability loops** — systems document themselves as they evolve.

Sustainable Software Engineering: Synthesis and Reflection

The evolution from fairness to sustainability can be seen as a chain of reinforcing principles:

Fairness → Transparency → Accountability → Governance → Sustainability

Each link strengthens the next:

- *Fairness* defines the ethical goal.
- *Transparency* makes fairness visible.
- *Accountability* ensures responsibility.
- *Governance* embeds responsibility into institutions.
- *Sustainability* ensures the system's ethics endure over time.

Together, they form a model of **socially sustainable software engineering**, where ethical reflection and technical design evolve in tandem.

Sustainable Software Engineering: References

Ethics, Accountability, and Governance Foundations

- ACM. (2018). *ACM Code of Ethics and Professional Conduct*. Association for Computing Machinery.
- Floridi, Luciano, et al. "AI4People—An ethical framework for a good AI society: Opportunities, risks, principles, and recommendations." *Minds and machines* 28.4 (2018): 689–707.
- Samarawickrama, Mahendra. "AI governance and ethics framework for sustainable AI and sustainability." *arXiv preprint arXiv:2210.08984* (2022).
- Canton, Helen. "Organisation for economic co-operation and development—OECD." *The Europa Directory of international organizations 2021*. Routledge, 2021. 677–687.
- Whittaker, Meredith, et al. *AI now report 2018*. New York: AI Now Institute at New York University, 2018.

Transparency and Explainability

- Doshi-Velez, Finale, and Been Kim. "Towards a rigorous science of interpretable machine learning." *arXiv preprint arXiv:1702.08608* (2017).
- Selbst, Andrew D., and Solon Barocas. "The intuitive appeal of explainable machines." *Fordham L. Rev.* 87 (2018): 1085.
- Kossow, Niklas, Svea Windwehr, and Matthew Jenkins. *Algorithmic transparency and accountability*. Transparency International, 2022.
- Diakopoulos, Nicholas. *Automating the news: How algorithms are rewriting the media*. Harvard University Press, 2019.

Sustainable Software Engineering: References

Law and Regulation (EU AI Act and Governance)

- European Union. (2024). *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 Laying Down Harmonised Rules on Artificial Intelligence (Artificial Intelligence Act)*. Official Journal of the European Union.
- Smuha, Nathalie A. "Beyond a human rights-based approach to AI governance: Promise, pitfalls, plea." *Philosophy & Technology* 34.Suppl 1 (2021): 91-104.

Governance Frameworks and Organizational Ethics

- Raji, Inioluwa Deborah, et al. "Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing." *Proceedings of the 2020 conference on fairness, accountability, and transparency*. 2020.
- Jobin, Anna, Marcello Lenca, and Effy Vayena. "The global landscape of AI ethics guidelines." *Nature machine intelligence* 1.9 (2019): 389-399.

Documentation for Accountability (Model Cards, Datasheets)

- Gebru, Timnit, et al. "Datasheets for datasets." *Communications of the ACM* 64.12 (2021): 86-92.
- Mitchell, Margaret, et al. "Model cards for model reporting." *Proceedings of the conference on fairness, accountability, and transparency*. 2019.
- <https://modelcards.withgoogle.com/>

Sustainable Software Engineering - Accountability, Transparency & Governance

